

PEC 1 - Visión Artificial

Introducción

El procesamiento digital de imágenes representa uno de los pilares fundamentales de la visión artificial, permitiendo analizar, transformar y extraer información útil a partir de datos visuales. En esta primera memoria de la asignatura, se recoge el trabajo realizado en la PEC 1 de Visión Artificial, donde el objetivo principal es familiarizarse con un conjunto amplio y representativo de técnicas esenciales de la librería OpenCV.

A lo largo del desarrollo de esta PEC se han abordado problemas reales y variados: eliminación de ruido, transformaciones geométricas lineales y no lineales, calibración de cámara, estimación de movimiento mediante flujo óptico, seguimiento de objetos y segmentación. Cada uno de estos bloques introduce conceptos diferentes en su mayoría facilitados en los tutoriales de la librería principal.

Siguiendo con la línea, el trabajo se ha implementado íntegramente en Python utilizando la librería OpenCV, lo que ha permitido aplicar toda la manipulación de imágenes en base a las guías de la misma. Igualmente la idea no era replicar los tutoriales, sino adaptarlos para problemas reales.

En forma de preludio, toda esta entrega permite recorrer un espectro completo que va desde el preprocesado básico hasta métodos avanzados de segmentación y reconstrucción geométrica. Y esta memoria se organiza de forma metódica, unidireccional y coherente, integrando análisis, justificación técnica y resultados visuales, con el objetivo de demostrar la comprensión de los métodos aplicados y su relevancia en distintos problemas de visión artificial.

Anexo de recursos gráficos

Las imágenes con los resultados no las incluí directamente por limitaciones de memoria, por lo que las he dejado únicamente referenciadas. Se encuentran dentro de la estructura del proyecto, en la carpeta resultados. En cada ejercicio, incorporo la referencia correspondiente a su imagen.

Ejercicio 1

Las imágenes del directorio “DibujosNPT” presentan distintos tipos de alteraciones: sombreado irregular del papel, variaciones de iluminación, manchas locales, trazos muy finos y ruido de alta y baja frecuencia. El objetivo del ejercicio es diseñar una aplicación interactiva que permita explorar de manera visual diferentes estrategias de limpieza y seleccionar, para cada caso, la combinación de operadores más adecuada, manteniendo siempre la forma aproximada de los trazos originales.

Para ello, se desarrolló una interfaz basada en OpenCV con Sliders que permiten ajustar dinámicamente cada etapa del preprocesado. El enfoque no consiste en aplicar una única transformación global, sino en construir un pipeline modular capaz de combinar la mayoría de los filtros a nuestra disposición.

El Pipeline de procesamiento consta de cuatro etapas clave:

1. Sustracción de fondo: Aplica Gaussian Blur suavizado, que se resta de la original para eliminar gradientes de iluminación y facilitar la binarización.
2. Filtrado de ruido: Combina el filtro Mediana (para ruido impulsivo) con el filtro Bilateral (preserva bordes) para una limpieza sin difuminar excesivamente el trazo.
3. Umbralización: Ofrece opciones Global Manual, Otsu y Adaptativa (Gaussiana), siendo esta última la más eficaz para iluminación irregular al calcular umbrales locales.
4. Operaciones morfológicas: Permite Apertura (eliminar puntos de ruido) y Cierre (consolidar y restaurar la continuidad de trazos finos), siendo el cierre crucial para el dibujo.

Como quinto apartado podríamos comentar que se incluyó una Selección de Región de Interés (ROI) ajustable para inspección y aplicación local de transformaciones para “tapar” elementos ruidosos que al ser más grandes que la figura, aplicando las técnicas anteriores borraremos antes la figura principal que el ruido.

La combinación más efectiva fue: Sustracción de fondo, Filtrado bilateral, Umbralización adaptativa, Cierre morfológico suave y ROI para tapar la base de la casa. Esta secuencia eliminó el sombreado y el ruido, manteniendo trazos uniformes.

“resultado/resultado_1.jpg” “resultado/resultado_1’.jpg”

Ejercicio 2a

El objetivo de este apartado fue aplicar dos transformaciones geométricas fundamentales sobre las imágenes zigzag.jpg y brainLabels.png:

1. reescalado a un tamaño uniforme de 200×200 píxeles y
2. rotación de 45° alrededor del centro de la imagen.

Para evitar la pérdida de información durante la rotación, se calcula automáticamente un bounding box mayor que el original. Esto deja que las esquinas rotadas queden dentro del nuevo marco sin recorte.

El proceso genera dos versiones finales de cada imagen, con rotaciones limpias, sin recorte y con fondo negro uniforme.

“resultado/resultado_2a.png”

Ejercicio 2b

En este apartado se trabajó con una transformación afín compuesta a partir de tres operaciones consecutivas aplicadas sobre la imagen zigzag.jpg:

- T1: Escalado a 1/4
- T2: Inclinación (shear) de -30°
- T3: Rotación de 90° hacia la izquierda

El objetivo es obtener la secuencia de imágenes intermedias (2.b.1.1) y, además, generar la misma transformación de forma equivalente en un único paso mediante la composición matricial correcta (2.b.1.2).

Al final, se resuelve también el ejercicio de transformación (2.b.2), donde se debe reposicionar una ROI rectangular aplicando una combinación matemática de transformaciones simples.

Pasos seguidos:

- T1 (Escala 1/4): Reduce la imagen al 25% manteniendo proporciones.
- T2 (Inclinación -30°): Aplica un shear horizontal. Requiere cálculo de nuevos límites y traslación compensatoria para evitar recortes por coordenadas negativas.
- T3 (Rotación 90°): Gira la imagen resultante, volviendo a requerir recalcular los límites y traslación para contener la figura no cuadrada.

La Transformación compuesta, (juntando todos los pasos a la vez) demuestra que la composición de transformaciones afines es afín y que el orden sí altera el resultado.

“resultado/resultado_2b1.png”

Finalmente, se resuelve el ejercicio 2.b.2 Reposicionamiento de una ROI diseñando primero a partir de código en base a las especificaciones del enunciado, donde aplicamos:

Traslación inversa (al origen), Rotación, Escalado opcional y Traslación final (al destino).

“resultado/resultado_2b2.png”

Ejercicio 2c1

La meta es transformar la imagen *zigzag.jpg* dividiéndola en dos mitades: la izquierda se comprime para ocupar un tercio del ancho final, y la derecha se expande al doble, ocupando los dos tercios restantes. Esto se logra con un reescalado independiente de cada mitad, ensamblándolas luego en un nuevo lienzo con la altura original.

Esta no es una transformación lineal global, ya que la relación coordenada entrada/salida varía por región, impidiendo el uso de una única matriz afín. Por tanto, es una deformación no lineal.

Para su implementación, se requiere remapeado por bloques o tratamiento regional, no siendo válido el operador `warpAffine`. El resultado es una imagen coherente donde el patrón zigzag se comprime a la izquierda y se estira a la derecha, sin afectar la verticalidad aplicando los 2 métodos: transformación afín por bloques y reescalado directo por mitades.

“resultado/resultado_2c1.png”

Ejercicio 2c2

En este apartado se aplicó la transformación donde todas las coordenadas están normalizadas entre 0 y 1:

$$\{ \{y' = y + 0.4 \cdot (x - 0.5)^2 - 0.1 \mid x' = x \} \}$$

La implementación se realiza mediante `cv. remap`, generando un campo de deformación que desplaza cada píxel según esta fórmula.

Esta transformación produce un doblamiento vertical que depende de la posición horizontal. La función cuadrática desplaza más los puntos cercanos a los extremos que los del centro, generando como resultado una imagen de ajedrez con una deformación parecida a un arqueamiento.

“resultado/resultado_2c2.png”

Ejercicio 3.1

En este apartado se realizó la calibración de la cámara usando las imágenes del tablero de ajedrez disponibles en el directorio `data/calib` especificadas para su uso en el enunciado. El objetivo es obtener los parámetros intrínsecos y de distorsión de la cámara, así como representar un cubo en 3D sobre el plano del tablero siguiendo el ejemplo en los tutoriales de OpenCV.

Proceso de Calibración:

1. Detección de esquinas: Se localizan y refinan las esquinas internas del tablero (7x6) (`findChessboardCorners` y `cornerSubPix`), asociando puntos 2D de la imagen con puntos 3D reales ($z=0$).
2. Cálculo de parámetros: Se aplica `calibrateCamera` para obtener la matriz intrínseca K , los coeficientes de distorsión y el error de reproyección.
3. Rectificación: Se verifica la corrección de la distorsión generando una imagen rectificada.

Representación 3D (Cubo):

1. Se detectan las esquinas en una imagen y se estima la pose de la cámara (`solvePnP`).
2. Se proyectan los puntos 3D del cubo en la imagen (`projectPoints`).
3. Se dibuja el cubo con la base amarilla, aristas verticales azules y cara superior verde.

Siguiendo estos pasos, el proceso permite obtener parámetros intrínsecos, verificar la distorsión y representar un objeto 3D con coherencia geométrica sobre el tablero con los colores solicitados. “resultado/resultado_3.1.png”

Ejercicio 3.2

Este apartado repitió el proceso de calibración con la cámara del móvil para obtener sus parámetros intrínsecos reales y el objetivo es modelar la proyección de un patrón plano y comparar la calidad de calibración con la cámara de la PEC.

Captura del patrón y detección de esquinas

Después de probar con múltiples opciones recopiladas en data/calib_movil, se tomaron imágenes manteniendo el patrón de ajedrez (3x7 esquinas internas) rígido pues fueron las que me funcionaron. Se usó findChessboardCorners para detectar la cuadrícula y cornerSubPix para refinar la posición. Cada imagen válida aportó puntos 2D y 3D correspondientes.

Obtención de los parámetros intrínsecos

Con suficientes pares de puntos, se ejecutó calibrateCamera, obteniendo la matriz intrínseca, los coeficientes de distorsión y el error de reproyección. Estos parámetros se guardaron en B_movil.npz para su uso futuro en otro ejercicio.

Corrección de distorsión (Undistort)

Para validar, se rectificó una imagen. Se calculó la matriz óptima con getOptimalNewCameraMatrix, se aplicó undistort y se recortó. La comparación visual demostró que la corrección eliminó la curvatura de las líneas, confirmando la compensación de la distorsión radial.

Resultado

La calibración de la cámara del móvil produjo un modelo geométrico consistente, con bajo error de reproyección y rectificación visualmente correcta. Esto confirma que la cámara del móvil puede modelarse eficazmente con parámetros intrínsecos estándar, la distorsión se corrige tras la rectificación, y el procedimiento es válido para cámaras distintas a las de la PEC, sin embargo el modelo es muy quisquilloso y a la mínima alteración en las imágenes o en la matriz de las esquinas internas dará error.

“resultado/resultado_3.2.png”

Ejercicio 3.3

Si un sistema de visión artificial puede detectar y localizar de forma fiable varios puntos significativos de la cabeza (ojos, nariz, barbilla, orejas...) como plantea el ejercicio, entonces es posible estimar hacia dónde apunta aplicando el mismo principio utilizado en el ejercicio 3.1: recuperar la pose 3D a partir de correspondencias entre puntos 2D detectados en la imagen y un modelo 3D sencillo del rostro, básicamente sacar el plano consecuente de la cara a partir de los puntos y definir el vector normal de este plano por ejemplo desde la nariz.

El procedimiento consiste en asignar a cada punto facial detectado su posición equivalente en un modelo 3D aproximado de una cabeza. Con estas correspondencias, el método Perspective-n-Point (PnP) permite calcular la pose de la cabeza, obteniendo los vectores de rotación y traslación respecto a la cámara. A partir del vector de rotación, basta con proyectar un eje frontal del modelo (por ejemplo, el eje Z partiendo de la nariz) sobre la imagen para determinar la dirección aproximada de la mirada: si apunta hacia arriba, abajo o hacia los lados.

Este enfoque, basado en tres etapas: detección 2D de puntos clave, modelo 3D simple y estimación PnP, es robusto y suficiente para obtener la orientación global de la cabeza sin necesidad de modelos faciales complejos.

“resultado/resultado_3.3.png”

Ejercicio 4.1

En esta parte se comparó dos métodos distintos para estimar el movimiento en vídeo. El primero fue Lucas–Kanade, que calcula el movimiento solo en puntos de interés, normalmente esquinas o zonas con textura. Este método funciona suponiendo que la intensidad no cambia mucho entre fotogramas y que el movimiento es pequeño. Por esa razón, no dio resultados en regiones suaves, porque allí no hay gradiente suficiente para que el algoritmo encuentre una solución única.

El segundo método fue el flujo óptico denso, que estima el movimiento en todos los píxeles de la imagen. Este enfoque genera un campo completo de movimiento y permite ver deformaciones o desplazamientos globales aunque la zona no tenga detalles. A cambio, es mucho más pesado computacionalmente y puede generar algo de ruido en áreas con poca textura, aunque siempre devuelve un vector de movimiento.

Para visualizar el flujo óptico se podrían dar estas 3 maneras:

- Vectores: se dibujan flechas sobre la imagen que muestran hacia dónde se mueve cada punto y cuánto se desplaza.
- Mapa de color (HSV): la imagen se colorea según el movimiento, el color indica la dirección y la intensidad del color muestra la velocidad.
- Warping: se “mueve” un frame siguiendo el flujo calculado para comprobar si el movimiento estimado es coherente.

Ejercicio 4.2

Para este ejercicio se implementó una aplicación simple de seguimiento basada en el algoritmo MeanShift (una versión intermedia de los 2 anteriores métodos comentados en el 4.1), y el objetivo es seleccionar manualmente la región inicial correspondiente a la mano y rastrearla en tiempo real usando la cámara interna.

Flujo (Protocolo):

1. El usuario selecciona la ROI azul inicial de la mano en un frame.
2. Se calcula el histograma de color de esa ROI, que sirve como modelo de color de la mano.
3. En cada frame, el seguimiento usa backprojection para localizar las zonas más probables y MeanShift desplaza la ventana a la región de mayor densidad de probabilidad.
4. La ventana ajustada se marca con un rectángulo verde.

Consideraciones:

- Requiere iluminación estable.
- La ROI inicial debe ser precisa (solo la mano).
- Colores de ropa u otras partes del cuerpo similares a la piel pueden causar ambigüedad.
- MeanShift no actualiza el modelo, por lo que grandes cambios de orientación o distancia pueden degradar el seguimiento (irse muy lejos, muy cerca, etc).
- El método solo desplaza la ROI según la similitud cromática, sin estimar forma o deformaciones. (Solo sigue la tonalidad del pixel guardado en el ROI así que, por ejemplo, si le doy al vuelta a la mano y tengo 2 tonalidades en mi mano, me dejaría de seguir)

Resultado: Seguimiento estable con iluminación uniforme y fondo no conflictivo. Es un ejemplo sencillo y eficaz de seguimiento basado en color que funcionará si se mantiene el sistema ideal.

“resultado/resultado_4.png”

Ejercicio 5.1

En este apartado se comparó métodos de umbralización (fija, Otsu y adaptativa) en tres imágenes con distintas dificultades (ruido, fondo irregular y texto).

Umbralización Global (Fija y Otsu)

- Fija (128): Solo útil con contraste claro. Falla con ruido o iluminación irregular.
- Otsu: Calcula el mejor umbral automáticamente, mejor que el fijo, pero al ser global, falla con iluminación no uniforme.
- Conclusión Global: Adecuada solo en imágenes con fondo limpio. Pierde detalle en imágenes ruidosas o con gradientes.

Umbralización Adaptativa

- Calcula un umbral distinto por región, compensando cambios de iluminación y sombras (explorando W y C). Invertir la imagen mejoró la separación con fondo claro.
- Resultados: Preservó mejor detalles finos (texto/bordes) y segmentó correctamente zonas no uniformes (fondo irregular), superando a los métodos globales incluso con ruido fuerte.

La umbralización global solo sirve con iluminación uniforme y contraste claro. La adaptativa resultó ser la más robusta, ajustando localmente y separando mejor los elementos en condiciones difíciles. En el código probé los métodos de umbralización sobre todas las imágenes del directorio data/threshold (TH_noisy2.png, TH_TE.gif y TH_scanTable.png), aplicando tanto umbralización global como adaptativa. Sin embargo, para ilustrar los resultados en la memoria utilicé únicamente la imagen TH_TE.gif, ya que es la que mejor muestra las diferencias entre los métodos y permite ver con claridad cómo se comporta cada tipo de umbralizado.

“resultado/resultado_5.1.png” “resultado/resultado_5.1’.png”

Ejercicio 5.2

Con umbralización local se podría intentar segmentar el rectángulo blanco de TH_noisy2.png o la pieza en forma de T de TH_TE.gif, pero no es una solución fiable por sí sola pues falta matizar.

La umbralización local decide si un píxel es blanco o negro mirando solo el brillo de los píxeles que tiene alrededor. Esto ayuda cuando la iluminación cambia en la imagen, pero tiene varios problemas:

- No sabe qué forma buscamos, solo mira niveles de gris.

- Si hay mucho ruido, pueden aparecer puntos blancos que no son del rectángulo o de la T.
- En zonas con poco contraste o bordes finos, partes del objeto se pueden perder o quedar rotas

Por eso, teóricamente, la umbralización local por sí sola no garantiza segmentar bien el rectángulo ni la T.

Lo razonable sería combinarla con otros pasos (por ejemplo, limpieza de ruido, operaciones morfológicas para cerrar huecos y detección de contorno, etc.) si se quisiera aislar bien esas figuras.

Ejercicio 5.3a

En este apartado se aplicó k-means sobre la imagen brain1.png para intentar separar, al menos de forma aproximada:

- la sustancia blanca (región más clara dentro del cerebro),
- la sustancia gris (región grisácea alrededor de la sustancia blanca),

teniendo en cuenta que también aparecen otras estructuras como el cráneo (muy brillante) y el fondo/CSF (muy oscuro) que habrá que evitarlos de alguna manera.

Metodología:

1. Cargué la imagen y la convertí a un vector de píxeles (intensidad únicamente, sin información espacial).
2. Probé $K = 3$, $K = 4$ y $K = 5$.
3. Para cada K , ejecuté cv.kmeans, obtuve y ordené los centroides por intensidad, y reconstruí la imagen segmentada.

Interpretación y Resultado:

Teóricamente, existen al menos 4 clases de intensidad (fondo/LCR, sustancia gris, sustancia blanca, cráneo).

- $K=3$ tiende a mezclar estructuras.
- $K=4$ ofrece la mejor separación (fondo, gris, blanco, cráneo/brillante), logrando el objetivo de distinguir mejor la sustancia blanca de la gris.
- $K=5$ resulta en una sobre-segmentación poco útil.

Conclusión: K-means sobre la intensidad de píxeles permite una segmentación aproximada de las principales estructuras cerebrales con $K=4$, aunque carece de contexto anatómico real (no soy médico).

“resultado/resultado_5.3a.png”

Ejercicio 5.3b

Se utilizó MeanShift para segmentar `jardin1.png` y `avion1.png`. A diferencia de la umbralización o k-means, MeanShift agrupa píxeles cercanos y de color similar, creando regiones más uniformes.

Parámetros:

- `sp` (radio espacial): Controla el suavizado por cercanía física.
- `sr` (radio de color): Controla el suavizado por similitud cromática.

Observaciones de los parámetros:

- Valores bajos (`sp/sr` pequeños): Poca segmentación, mucho detalle.
- Valores medios: Segmentación equilibrada, regiones limpias.
- Valores altos (`sp/sr` grandes): Segmentación muy agresiva, pérdida de detalle.

Ejemplo Jardín: Después del filtrado MeanShift, apliqué un umbral (Otsu) para obtener una segmentación binaria.

Ejemplo Avión: No se aplicó umbralización; se evaluó la segmentación cromática directa.

Resumen de la influencia:

- `sp` pequeño + `sr` pequeño → Ruido visual.
- `sp` medio + `sr` medio → Segmentación equilibrada.
- `sp` grande + `sr` grande → Pérdida de detalle, alta uniformidad.

Diferencia con MeanShift en Tracking (apartado 4.2):

El MeanShift en el 4.2 sigue una ventana basada en un histograma de color (modelo de la mano). No segmenta, al contrario que “aquí” en la 5.3b el MeanShift suaviza y agrupa píxeles de toda la imagen por color, siendo un paso previo a la segmentación de regiones grandes.

“resultado/resultado_5.3b.png” “resultado/resultado_5.3b’.png”

Ejercicio 5.4

En este apartado se aplicó el algoritmo Watershed para separar los lóbulos temporales y central de la región central del cerebro, aprovechando el conocimiento de dominio (sustancia blanca rodeada por gris).

1. Máscara inicial: Se generó una máscara aproximada del cerebro mediante umbralización y se seleccionó la componente conectada más grande, después de reescalar la imagen para que los filtros que le fuéramos a aplicar luego se hicieran mejor.
2. Preparación de semillas: El intento inicial de usar la sustancia blanca como semilla de foreground seguro resultó en una segmentación demasiado limitada pues solo cogían la sustancia blanca en vez de la gris. Por ello, se optó por un enfoque más robusto a mi parecer: generar semillas mediante erosión fuerte de la máscara del cerebro (incluyendo parte de la sustancia gris), y el *background seguro* mediante dilatación. La diferencia entre ambas definió la zona desconocida.
3. Ejecución de Watershed: Se aplicó Watershed a la imagen con las semillas preparadas, creando etiquetas que dividen el cerebro.
4. Clasificación: Tras Watershed, se erosiona la máscara para obtener una zona interior con el planteamiento de donde al ser los elementos que queremos fijar los más grandes, al erosionar la imagen serían los que quedarían. Las etiquetas con mayor solape con esta zona se clasificaron: las dos inferiores laterales como lóbulos temporales (verde y azul) y el resto como área central (rojo).

Conclusiones: Watershed es útil para separar regiones contiguas con intensidades similares. La clave es el uso de semillas adecuadas; las erosiones fuertes produjeron semillas más amplias y una segmentación más útil que el uso directo de la sustancia blanca. La clasificación final se basó en la posición anatómica y el solape con la zona interior.

“resultado/resultado_5.4.png”